8. Technology Stack and Design Decisions

The Process Change Impact Simulator will be developed using Python and Flask. The selected technologies support the main project requirements, including process modeling, dependency analysis, simulation, data storage, and visualization.

Technology	Purpose and design decision
Python 3.12	Used for application logic, graph analysis, and simulation. Python provides suitable libraries for processing structured data and implementing graph algorithms.
Flask	Used as the web framework to connect user interfaces with Python modules. Flask is lightweight and allows the application to be organized into separate components.
HTML and CSS	Used to create web pages for process creation, change selection, and results. These technologies support a traditional browser-based interface.
SQLite	Used to store processes, activities, dependencies, proposed changes, and simulation results. It is suitable for a student project because it requires minimal setup and does not need a separate database server.
NetworkX	Used to represent workflows as directed graphs, trace downstream dependencies, identify cycles, and analyze connected activities.
Pandas and NumPy	Used to organize process data and perform calculations or comparisons required by the simulation.
Plotly and Matplotlib	Used to display dependency graphs and compare original and modified process results. Plotly can support interactive visualizations, while Matplotlib can generate static charts.
PyYAML and JSON	JSON will support process import and export. PyYAML may be added if YAML-based process definitions are needed.
pytest	Used to test graph construction, dependency analysis, simulation results, and error handling.
Visual Studio Code and Git	VS Code will be the main development environment, while Git will track source-code changes and support version control through GitHub.

Important design decisions

- Flask instead of Streamlit: Streamlit was considered in the original proposal. Flask was selected for implementation because it provides greater flexibility in designing separate web pages, routes, and application modules.
- SQLite instead of MySQL or PostgreSQL: SQLite is sufficient for storing the project's sample workflows and simulation results. A server-based database may be considered if the application later needs multiple concurrent users or larger deployments.
- Graph algorithms instead of machine learning: NetworkX will perform the main impact analysis because process dependencies are represented explicitly. The system will trace connected activities and compare process execution paths rather than rely on predictions from a trained model.
- Synthetic data instead of organizational records: Sample onboarding and purchase approval workflows will be created using publicly documented process characteristics. This avoids the need for confidential business data.

- Local development before hosting: The application will initially run locally on a Windows computer. Cloud hosting is not required for the first implementation.